

How to Install Images

Introduction

This guide assumes the use of the provided Multiboot_Admin environment to install additional images, although the same approach is used to install images using an external Linux os. The Multiboot_Admin os should auto-login with two console screens but in a gui environment. For info **user=pi, password=pi**

1) Install or update the Multiboot Toolchain

- If using the Multiboot_Admin os then this toolchain comes pre-installed at /home/pi/Pi_Multiboot. **However you should make sure that it is up to date** by running

```
cd ~  
./update_imager.sh
```

- If using an external os you can fetch and then run the same script

```
cd ~  
wget https://raw.githubusercontent.com/AndyDeSheffield/Pi_Multiboot/refs/heads/main/tools/update_imager.sh  
sudo chmod +x update_imager.sh  
./update_imager.sh
```

2) Mount the images partition which will receive your new os

- When using the Multiboot_Admin os this is most likely your current images partition. In this case it can be mounted using the script in the Pi_Multiboot/tools directory

```
Pi_Multiboot/tools/mountdrives.sh
```

your target partition will be **/mnt/realimages**

- Otherwise you will likely be installing to an external usb key partition (example /dev/sdb3) so

```
mkdir -p ~/realimages  
sudo mount /dev/sdb3 ~/realimages
```

your target partition is of course ~/realimages

3) Run the image creation tool

- First check the help
- Note that after each action the imaging tool gives the option to continue or abort (which backs out previous steps)

```
cd ~/Pi_Multiboot/create_images  
./install_image.sh -h
```

```
Usage: $0 -r=<root_mount> -n=<name_of_image> -m=<pi_model> -s=<size> [-e] [-h]  
-r=<root_mount>      Root of mounted image disk (e.g. -r=/mnt/realimages)  
-n=<name>            Name of image (letters, numbers, '-', '_')  
-m=<pi_model>        Pi model (e.g. pi3b,pi3b+,pi4, pi5)  
-s=<size>            Image size (e.g. 10G)  
-c                  Apply custom files to the image directory (will not hurt if there are no custom files)  
-e                  Expand root filesystem (only if exactly 2 partitions exist)
```

-h Show this help

Note that the script needs to be run with sudo and relies on the device tree compiler (DTC) being available

-
- Run the tool as sudo with the appropriate parameters

```
sudo ./install_image.sh -r=/mnt/realimages -n=My_New_OS -m=pi4 -s=10G -e -c
```

1. Step 1 creates the blank .img file on the root Mount

Step 1: Creating target directory and blank image...

Created image: /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4.img (size 10G)

Step 1 completed. Continue (c) or Abort (a)? [c/a]:

1. Step 2 runs a modified version of the raspi-imager that accepts loop mounts as targets

Continuing...

Step 2: Loop-mounting image and running Raspberry Pi Imager...

Loop device: /dev/loop2

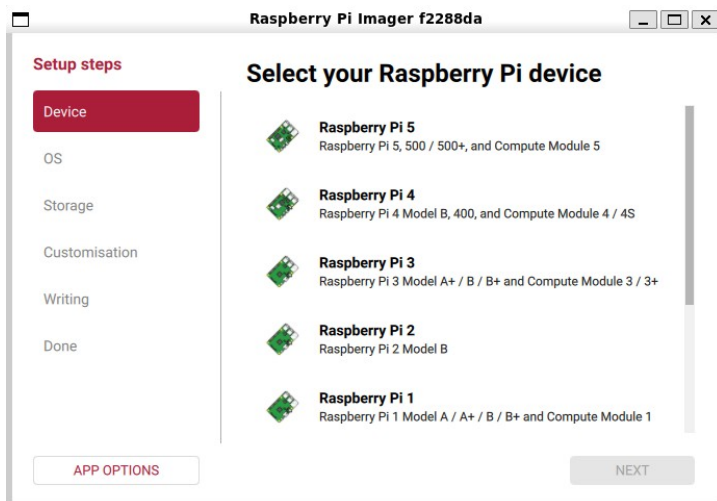
Launching Raspberry Pi Imager...

Refreshing loop partitions...

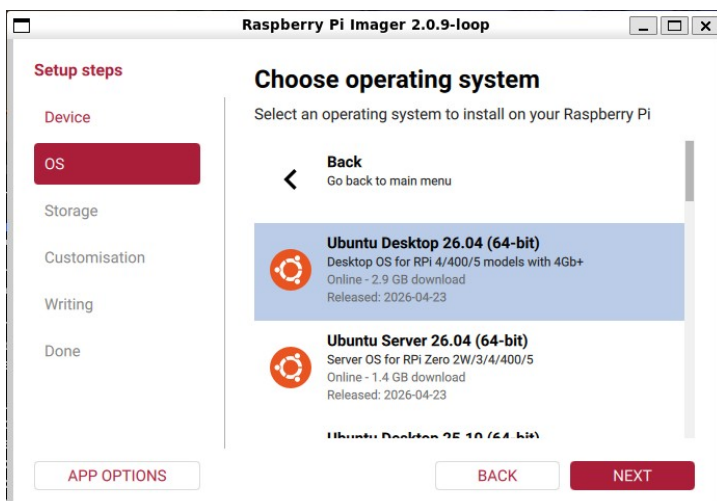
Reattached loop device: /dev/loop2

Follow the usual imaging process selecting the loop device attached to your image as the target

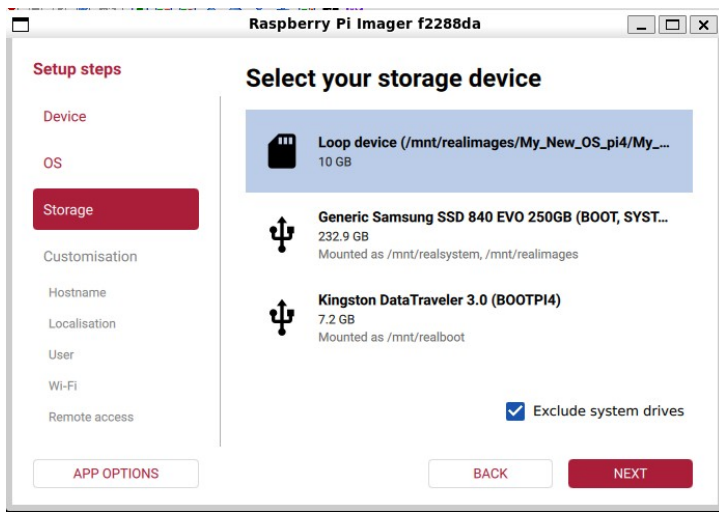
- Select Pi model



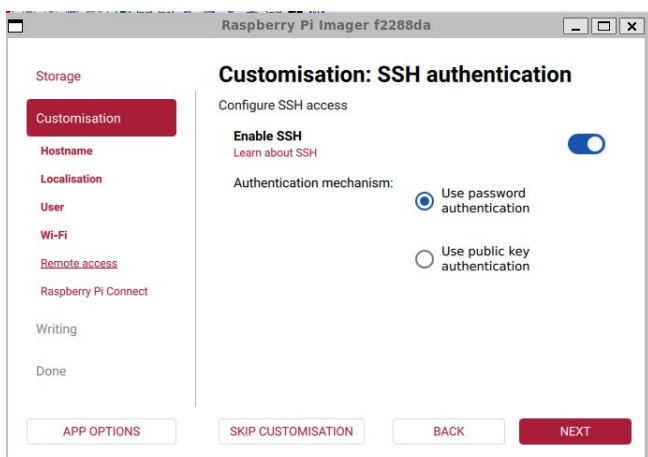
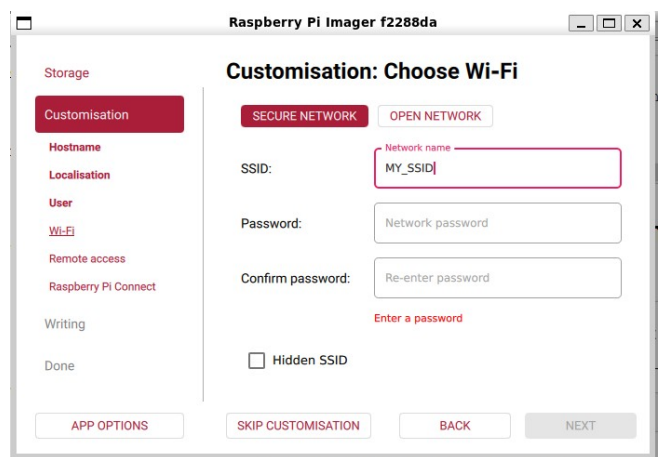
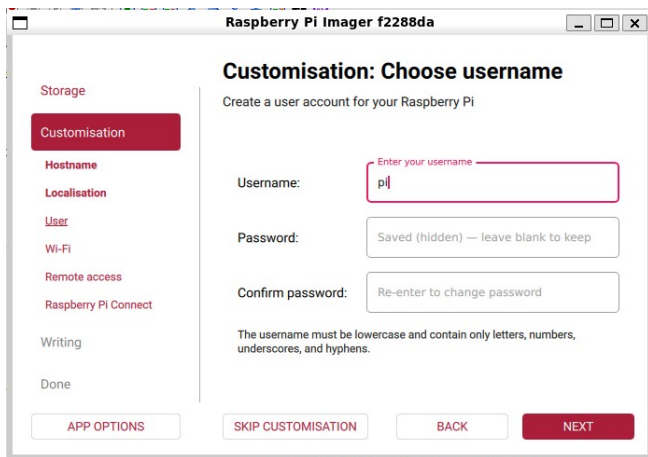
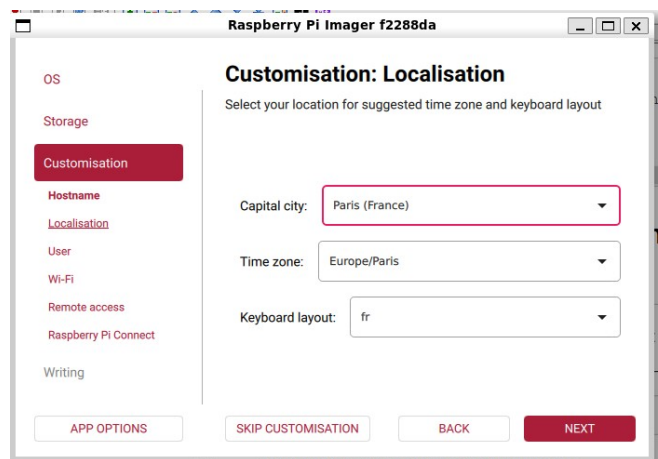
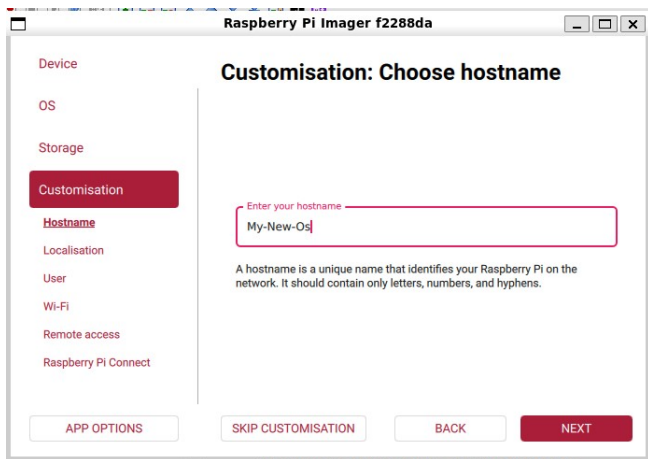
- Select Operating System



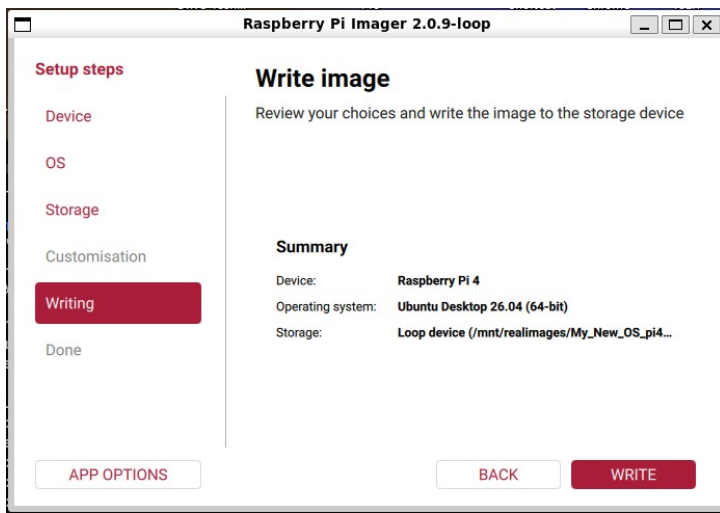
Select Target loop --- **Be careful NOT to select the SSD or Hard drive itself!**



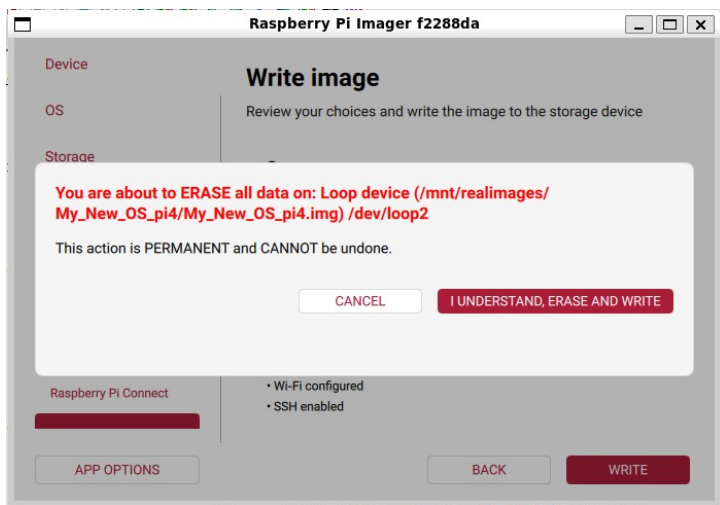
- Configure if desired



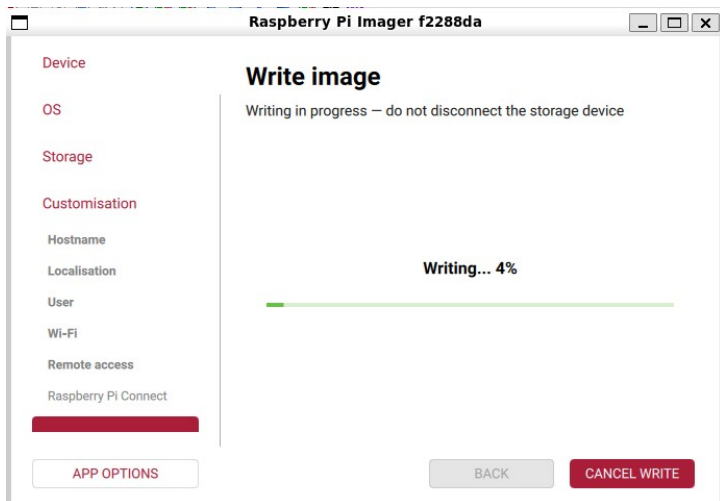
- Write Image



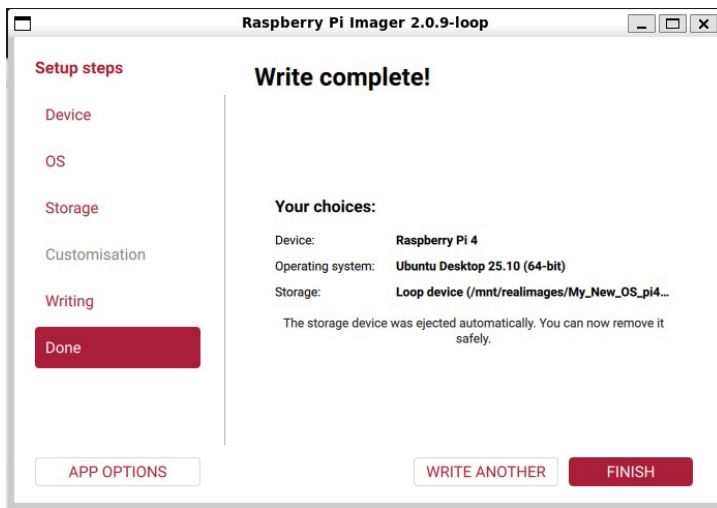
- Confirm Write



- Writing



- Finish



Step 2 completed. Continue (c) or Abort (a)? [c/a]:

1. Step 3 creates a dtb file with all the overlays and properties specified in the config.txt of the target boot partition merged in.
 - It copies that dtb, along with the target kernel and any initramfs into a directory /target_name on the root mount.
 - A suitable Grub entry file is also created
 - The kernel cmdline used in the official image is extracted for reference. Use the "extras" variable in the grub config to add any desired values.
 - The config.txt from the original boot partition is extracted for reference and a file "untreated_config.txt" is created showing lines that it has not been possible to process.
 - If the -e option is selected the root partition of the new image will be expanded to fill the img file
 - If the -c option is selected then custom files for the image will be copied into the directory where the img file is hosted. Details of what will be copied are displayed and you can confirm the copy before it is made

Continuing...

Step 3: Extracting DTB and kernel...

Handling uefi fixes. Installing cpufix.dtbo, dmafix.dtbo and uefi_fixes.txt

Model: pi4 (index 5)

Boot directory: ./bootpart

Will output to : /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4{.dtb,.kernel,.cmdline,.initrd}

Any kernel, cmdline and initrd files will be copied (-x specified.)

UNTREATED CONFIG LINES stored in /mnt/realimages/My_New_OS_pi4/untreated_config.txt

uefi_fixes.txt found and will be processed after config.txt

COPYING FULL Config.txt ./bootpart/config.txt to /mnt/realimages/My_New_OS_pi4/config.txt

COPYING SPECIFIED KERNEL ./bootpart/current/vmlinuz to /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4.kernel

COPYING INITRAMFS ./bootpart/current/initrd.img to /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4.initrd

SET_BASE_PROPERTY (group 0): audio=on

SET_BASE_PROPERTY (group 1): i2c_arm=on

SET_BASE_PROPERTY (group 2): spi=on

SET_BASE_PROPERTY (group 3): watchdog=on

Warning: unknown base dtparam 'watchdog' ignored

LOAD_OVERLAY (group 4): ./bootpart/current/overlays/vc4-kms-v3d-pi4.dtbo

APPLY_OVERLAY (group 4): ./bootpart/current/overlays/vc4-kms-v3d-pi4.dtbo

LOAD_OVERLAY (group 5): ./bootpart/current/overlays/dwc2.dtbo

APPLY_OVERLAY (group 5): ./bootpart/current/overlays/dwc2.dtbo

LOAD_OVERLAY (group 6): ./bootpart/current/overlays/miniuart-bt.dtbo

APPLY_OVERLAY (group 6): ./bootpart/current/overlays/miniuart-bt.dtbo

LOAD_OVERLAY (group 7): ./bootpart/current/overlays/cpufix.dtbo

APPLY_OVERLAY (group 7): ./bootpart/current/overlays/cpufix.dtbo

LOAD_OVERLAY (group 8): ./bootpart/current/overlays/dmafix.dtbo

```

APPLY_OVERLAY (group 8): ./bootpart/current/overlays/dmafix.dtbo
COPYING MERGED DTB to /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4.dtb
Files successfully created
Step 3.7: Optional expansion...
Expanding root filesystem...
Model: Loopback device (loopback)
Disk /dev/loop2: 10.7GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

```

Number	Start	End	Size	Type	File system	Flags
1	1049kB	538MB	537MB	primary	fat32	boot, lba
2	538MB	8922MB	8384MB	primary	ext4	

```

e2fsck 1.47.2 (1-Jan-2025)
Pass 1: Checking inodes, blocks, and sizes
Pass 2: Checking directory structure
Pass 3: Checking directory connectivity
Pass 3A: Optimizing directories
Pass 4: Checking reference counts
Pass 5: Checking group summary information

```

```

writable: ***** FILE SYSTEM WAS MODIFIED *****
writable: 121987/512064 files (0.1% non-contiguous), 1377596/2046785 blocks
resize2fs 1.47.2 (1-Jan-2025)
Resizing the filesystem on /dev/loop2p2 to 2490112 (4k) blocks.
The filesystem on /dev/loop2p2 is now 2490112 (4k) blocks long.

```

```

Expansion complete.
[cleanup] Forcing release of ./bootpart...
Generating GRUB entry: /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4_grub.cfg
GRUB entry written to: Grub_My_New_OS_pi4.cfg
Running customisation (may overwrite generated grub entry)...
  sudo ./deploy_custom_files.py /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4.img
[detect] OS: ubuntu, VERSION_ID: 25
[info] OS=ubuntu version=25 (numeric 25.0)
[info] UUIDs: {1: '0808-11DE', 2: 'c418dc3c-288d-475d-b6d2-48008dea7829'}
[match] Using files from: /home/pi/Pi_Multiboot/create_images/custom_files/ubuntu/25+

```

```

  Custom files from : /home/pi/Pi_Multiboot/create_images/custom_files/ubuntu/25+
  Will be copied to : /mnt/realimages/My_New_OS_pi4
  Prefixed with      : My_New_OS_pi4_

```

```

Continue? [Y/n]: Y
[deploy] Deployed 2 file(s) to /mnt/realimages/My_New_OS_pi4
[done] Complete.

```

4) Insert the contents of the suggested Grub file into grub.cfg

- If using the Multiboot_Admin os and you have mounted the target partition using mountdrives.sh you are already in a position to do this.
- Otherwise you have to insert the device that you will be using and mount it. The grub.cfg file that needs updating will be /efi/boot/grub.cfg
- Note that it may, in any case be preferable to update /mnt/realboot/efi/boot/grub.cfg using a text editor like nano, rather than the commands below, especially if you wish to position the entry in the grub menu

```

sudo cp /mnt/realboot/efi/boot/grub.cfg /mnt/realboot/efi/boot/grub.cfg.bak
sudo cat /mnt/realimages/My_New_OS_pi4/My_New_OS_pi4_grub.cfg | sudo tee -a /mnt/realboot/efi/boot/grub.cfg

```

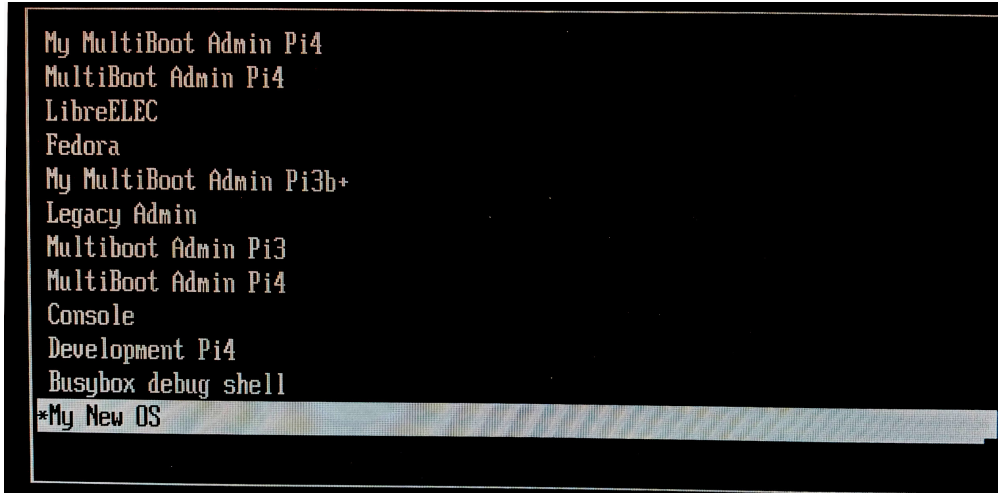
5) Unmount the partitions cleanly

With the Multiboot_Admin os use

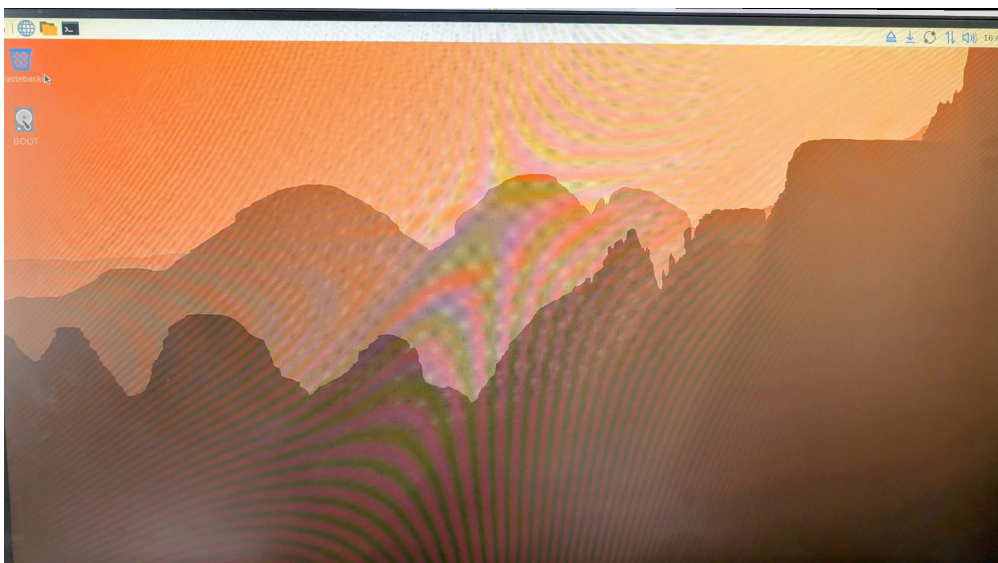
```
~/Pi_Multiboot/tools/unmountdrives.sh
```

6) Test

- Grub entry



- After Boot



- Note that
 - First boot of many OSes can take a long time
 - There will be some errors in the startup mainly due to upower (not sure what this is)
 - There is often an unexpected reboot after the very first boot. This may be the Raspbian resize functionality kicking in, I'm not sure.

Conclusion

All the 64 bit Raspbian images and the 64 bit Ubuntu images (<v26) that I've tested booted using this technique. All 4 processors are present and there is working sound but that's all that I've tested.

Footnote

Using some different variations I've also managed to boot, but which use (included) customisation :-

- **LibreELECT** (v21.3 and only for the pi4 currently, due to lack of pi5 hardware and LibreELECT for the pi3 being 32 bit)
 - Use the same process as above to create the image
 - A custom initrd will be added to the boot process to create the image loop mount.
 - A custom grub entry will be created
- **Fedora** (Fedora-Workstation-Disk-43-1.6.aarch64.raw.xz)
 - [From the official site] (<https://fedoraproject.org/workstation/download/>)
 - This also needs a specific grub entry and initramfs overlay.
 - The build process is a bit more manual. I'll provide details if anyone is interested
 - A custom _extra initrd will be added to the boot process
 - A custom grub entry will be created
- **Lineage** (-23.2-20260128-UNOFFICIAL-KonstaKANG-rpi4 (includes booting to twrp as a seperate grub entry)
 - See seperate document (to be added)
 - A custom _extra initrd will be added to the boot process
 - A custom grub entry will be created
- **Ubuntu 26+**
 - The Ubuntu team has moved to a much more complex boot process based on dracut (which I passionately hate!).
 - Use the standard process to install the image (with my version of Raspi-Imager)
 - A custom _extra initrd will be added to the boot process
 - A custom grub entry will be created
- **Windows**(Version 22H2 for the pi4 only)
 - This is Windows from the [Windows on Raspberry](#) project, which I believe has been discontinued in favour of an alternative approach using a vm. Windows is native booted from a vhd using ntloader.arm64 from the [ntloader project](#) . Unfortunately, of course, the windows.vhd has to be installed on an NTFS or ExFAT partition.
 - If I was able to publish my Windows vhd without infringing MS licensing this would be an easy install. As it is, you have to go through a number of different steps, using a variety of tools. I'll publish the process if anyone is interested.